HANDS-ON #03 — Guia técnico
Testes unitários com xUnit

Ponto de Conhecimento Tecnológico
C# · .NET 10 · Visual Studio 2026

Guia de reprodução do exercício: configuração da solução, escrita de testes, execução, recolha e interpretação da cobertura. Exemplo: uma calculadora de descontos com três métodos de negócio.

1. Tecnologias e responsabilidades

Tecnologia	Função
.NET 10 / C#	Plataforma e linguagem da aplicação e dos testes.
Visual Studio 2026	IDE para criar, editar, compilar e depurar a solução.
xUnit 2.9.3	Framework de testes: Fact, Theory, InlineData e asserções.
Microsoft.NET.Test.Sdk 17.14.1	Infraestrutura de execução e descoberta dos testes.
xunit.runner.visualstudio 3.1.4	Adaptador para integração com o Test Explorer / VSTest.
coverlet.collector 6.0.4	Recolha de cobertura durante a execução dos testes.
ReportGenerator 5.5.12	Conversão de ficheiros Cobertura XML em relatório HTML.

2. Estrutura da solução

```
TestesUnitariosHandsOnDemo/
  TestesUnitariosHandsOnDemo.slnx
  TestesUnitariosHandsOnDemo.Core/
    CalculadoraDescontos.cs
  TestesUnitariosHandsOnDemo/
    UnitTest1.cs
  CoverageReport/
  README.md
  .gitignore
```

O projeto Core é uma biblioteca de classes (.NET 10). O projeto de testes usa xUnit (.NET 10) e referencia o projeto Core. No Visual Studio: criar a solução, adicionar a biblioteca de classes, adicionar o projeto xUnit e, no projeto de testes, usar Add → Project Reference para selecionar Core.

3. Configurar o projeto de testes

O ficheiro .csproj do projeto de testes deve identificar net10.0, os pacotes NuGet e a referência ao projeto Core. O exemplo seguinte corresponde à configuração partilhada no exercício:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <IsPackable>false</IsPackable>
```

```
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="coverlet.collector" Version="6.0.4" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.14.1" />
    <PackageReference Include="xunit" Version="2.9.3" />
    <PackageReference Include="xunit.runner.visualstudio" Version="3.1.4" />
  </ItemGroup>
  <ItemGroup>
    <ProjectReference Include="..\TestesUnitariosHandsOnDemo.Core\
TestesUnitariosHandsOnDemo.Core.csproj" />
    <Using Include="Xunit" />
  </ItemGroup>
</Project>
```

Depois de alterar o projeto: guardar, restaurar pacotes NuGet e compilar a solução. As versões acima são as usadas neste exercício, não uma recomendação universal de versões mais recentes.

4. Regras de negócio implementadas

CalcularSubtotal(precoUnitario, quantidade): rejeita preço negativo e quantidade menor ou igual a zero; devolve preço × quantidade.

CalcularDesconto(subtotal, percentagem): rejeita subtotal negativo e percentagem fora do intervalo 0–100; devolve subtotal × percentagem / 100.

CalcularTotal(subtotal, desconto): rejeita valores negativos e desconto superior ao subtotal; devolve subtotal − desconto.

5. Criar os testes com xUnit

Os testes seguem o padrão AAA: Arrange (preparar dados), Act (executar método), Assert (comparar resultado). Os exemplos abaixo são ilustrativos e podem ser adaptados ao ficheiro UnitTest1.cs.

```
using TestesUnitariosHandsOnDemo.Core;
using Xunit;

public class CalculadoraDescontosTests
{
    private readonly CalculadoraDescontos _sut = new();

    [Fact]
    public void CalcularSubtotal_ValoresValidos_DevolveSubtotal()
    {
        // Arrange
        decimal preco = 12.50m;
        int quantidade = 2;
        // Act
        decimal resultado = _sut.CalcularSubtotal(preco, quantidade);
        // Assert
        Assert.Equal(25.00m, resultado);
    }

    [Theory]
    [InlineData(100, 10, 10)]
    [InlineData(80, 25, 20)]
    public void CalcularDesconto_ValoresValidos_DevolveDesconto(
        int subtotal, int percentagem, int esperado)
    {
```

```
        Assert.Equal(esperado,
            _sut.CalcularDesconto(subtotal, percentagem));
    }

    [Fact]
    public void CalcularTotal_DescontoSuperiorAoSubtotal_LancaExcecao()
    {
        Assert.Throws<ArgumentException>(() =>
            _sut.CalcularTotal(10m, 20m));
    }
}
```

[Fact] testa um caso; [Theory] com [InlineData] permite testar vários conjuntos de entradas; Assert.Equal compara valores; Assert.Throws<ArgumentException> confirma a rejeição de dados inválidos.

6. Executar os testes

Na pasta da solução, abrir Developer PowerShell e executar:

```
dotnet restore
dotnet build
dotnet test
```

No exercício, o terminal apresentou 26 testes aprovados, zero falhados. A contagem refere-se à execução documentada e não é uma propriedade automática do xUnit.

7. Recolher a cobertura de código

```
dotnet test --collect:"XPlat Code Coverage"
```

O Coverlet gera um ficheiro coverage.cobertura.xml dentro de TestResults, numa subpasta identificada por execução. O ficheiro XML pode ser consultado para validar as métricas.

8. Gerar e consultar o relatório HTML

```
dotnet tool install --global dotnet-reportgenerator-globaltool

reportgenerator "-reports:TestesUnitariosHandsOnDemo\TestResults\**\coverage.cobertura.
xml" "-targetdir:CoverageReport" "-reporttypes:Html"
```

Abrir CoverageReport/index.html no navegador. A utilização de ** pode agregar vários relatórios de execuções anteriores; para um resultado estritamente isolado, apontar para o XML da execução pretendida.

9. Resultados e interpretação

Indicador	Resultado documentado
Testes	26 aprovados / 0 falhados
Cobertura de linhas	19/19 linhas instrumentadas = 100%
Cobertura de ramificações	16/16 ramificações instrumentadas = 100%
Classe	CalculadoraDescontos (3 métodos)

A cobertura de 100% indica que as linhas e ramificações instrumentadas foram executadas. Não prova ausência de defeitos, nem substitui cenários bem escolhidos, testes de integração ou revisão de

requisitos. A métrica aplica-se à classe incluída no relatório, não a qualquer aplicação maior.

10. Diagnóstico: testes não descobertos no Test Explorer

Durante o exercício, o Test Explorer apresentou “Test discovery aborted” e zero testes, apesar de a compilação ter sido concluída. O comando dotnet test no Developer PowerShell executou os testes com sucesso. Este comportamento sugere um problema na integração de descoberta/execução do IDE, mas não permite determinar a causa exata sem os registos completos.

Sequência de diagnóstico recomendada: (1) confirmar compilação sem erros; (2) verificar pacotes xUnit, Test SDK e adaptador; (3) confirmar a referência ao projeto Core; (4) executar dotnet test; (5) atualizar o Test Explorer e reiniciar o IDE se necessário; (6) consultar o output detalhado da descoberta de testes. Não alterar permissões de executáveis do Visual Studio sem evidência de um problema de acesso.

11. Publicação e boas práticas

Publicar código-fonte, solução, README e .gitignore. O relatório CoverageReport pode ser incluído como artefacto demonstrativo, identificando-o como resultado de uma execução específica. Excluir normalmente bin/, obj/ e TestResults/ para evitar ficheiros temporários e resultados de compilação.

Repositório: https://github.com/FgAraujo-web/HandsOn-03-Testes-Unitarios-xUnit

Ponto de Conhecimento Tecnológico — Explorar · Experimentar · Aprender · Partilhar